Neighbourhood Processing

Neighbourhood processing

Contents

Introduction.

- We have seen in chapter 2 that an image can be modified by applying a particular function to each pixel value.

- Neighbourhood processing may be considered as an extension of this, where a function is applied to a neighbourhood of each pixel.

- We deal in this chapter with the mask and filter then

What is the mask and filter???

Introduction-cont

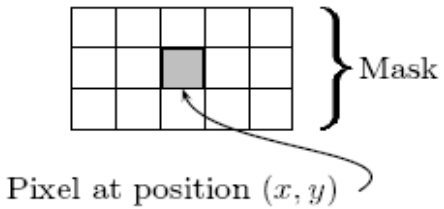
What is the mask???

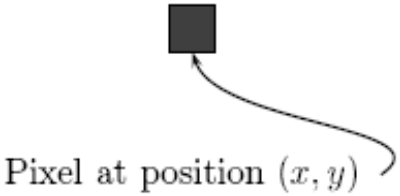
Is a rectangle (usually with sides of odd length) or other shape over the given image.

What is the filter???

The combination of mask and function

- If the function by which the new grey value is calculated is a linear function of all the grey values in the mask, then the filter is called a linear filter.





Introduction-cont

- *A linear filter* **can be implemented by multiplying all elements in the mask by corresponding elements in the neighbourhood, and adding up all these products.**
- **Ex:**
- **Suppose the mask 3*5 and it's values:**

$m(-1,-2)$	$m(-1,-1)$	$m(-1,0)$	$m(-1,1)$	$m(-1,2)$
$m(0,-2)$	$m(0,-1)$	$m(0,0)$	$m(0,1)$	$m(0,2)$
$m(1,-2)$	$m(1,-1)$	$m(1,0)$	$m(1,1)$	$m(1,2)$

Pixel values

$p(i-1,j-2)$	$p(i-1,j-1)$	$p(i-1,j)$	$p(i-1,j+1)$	$p(i-1,j+2)$
$p(i,j-2)$	$p(i,j-1)$	$p(i,j)$	$p(i,j+1)$	$p(i,j+2)$
$p(i+1,j-2)$	$p(i+1,j-1)$	$p(i+1,j)$	$p(i+1,j+1)$	$p(i+1,j+2)$

We now multiply and add:

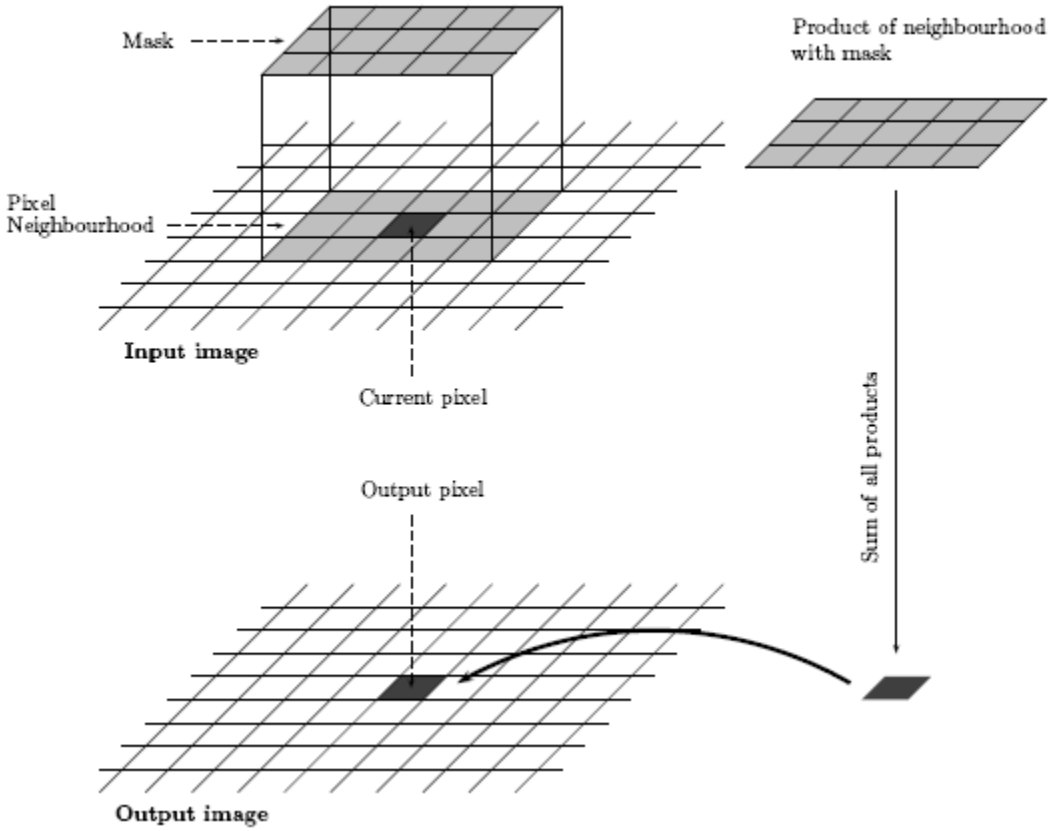
$$\sum_{s=-1}^{1}\sum_{t=-2}^{2} m(s,t)p(i+s,j+t).$$

What is the steps of Spatial filtering?

1) position the mask over the current pixel.

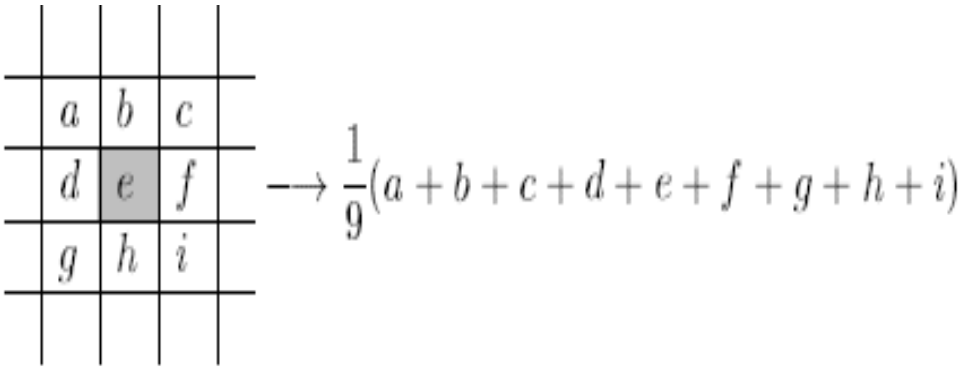
2) form all products of filter elements with the corresponding elements of the neighbourhood.

3) add up all the products.


Mask
Product of neighbourhood with mask
Pixel Neighbourhood
Input image
Current pixel
Sum of all products
Output pixel
Output image

Example:

Consider 3*3 mask we can do linear filter by take the average of all values in the mask.

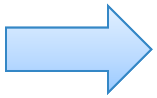


To apply this in mat lab write:

```
>> x=uint8(10*magic(5))

x =

   170   240    10    80   150
   230    50    70   140   160
    40    60   130   200   220
   100   120   190   210    30
   110   180   250    20    90
```



170	240	10	80	150
230	50	70	140	160
40	60	130	200	220
100	120	190	210	30
110	180	250	20	90

```
>> mean2(x(1:3,1:3))

ans =

   111.1111
```

Notation

- We can describe linear filter simply in terms of the coefficients of all the grey values of pixels within the mask.
- EX: the filter

$$\begin{bmatrix} 1 & -2 & 1 \\ -2 & 4 & -2 \\ 1 & -2 & 1 \end{bmatrix}$$

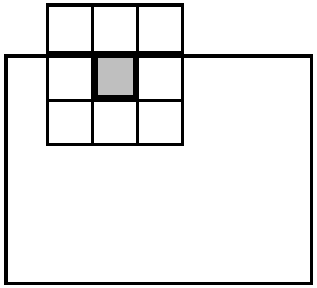
would operate on grey values as

	a	b	c
	d	e	f
	g	h	i

$$\longrightarrow a - 2b + c - 2d + 4e - 2d + g - 2h + i$$

Edges of the image

- There is an obvious problem in applying a filter
- what happens at the edge of the image, where the mask partly falls outside the image???!!!



Solutions:

1) Ignore the edges:
   - the mask is only applied to those pixels in the image for with the mask will lie fully within the image
   - As the result of this output image is smaller than the original.
   - If the mask is very large, a significant amount of information may be lost by this method.
2) Pad with zeros:
   - all values outside the image are zero.
   - output image of the same size as the original.
   - But it may consider unwanted artifacts like edges around the image

Filtering in Matlab:

- The filter2 function does the job of linear filtering:

  filter2(filter, image, shape)

- filter2(filter, image, 'same') is the default it produces a matrix of equal size to the original image matrix. It uses zero padding:

```
>> a=ones(3,3)/9

a =

    0.1111    0.1111    0.1111
    0.1111    0.1111    0.1111
    0.1111    0.1111    0.1111

>> filter2(a,x,'same')

ans =

   76.6667   85.5556   65.5556   67.7778   58.8889
   87.7778  111.1111  108.8889  128.8889  105.5556
   66.6667  110.0000  130.0000  150.0000  106.6667
   67.7778  131.1111  151.1111  148.8889   85.5556
   56.6667  105.5556  107.7778   87.7778   38.8889
```

```
>> filter2(a,x,'valid')

ans =

   111.1111   108.8889   128.8889
   110.0000   130.0000   150.0000
   131.1111   151.1111   148.8889
```

filter2(filter,image,'valid') applies the mask only to inside pixels.
The result will always be smaller than the original:

```
>> filter2(a,x,'full')

ans =

   18.8889   45.5556   46.6667   36.6667   26.6667   25.5556   16.6667
   44.4444   76.6667   85.5556   65.5556   67.7778   58.8889   34.4444
   48.8889   87.7778  111.1111  108.8889  128.8889  105.5556   58.8889
   41.1111   66.6667  110.0000  130.0000  150.0000  106.6667   45.5556
   27.7778   67.7778  131.1111  151.1111  148.8889   85.5556   37.7778
   23.3333   56.6667  105.5556  107.7778   87.7778   38.8889   13.3333
   12.2222   32.2222   60.0000   50.0000   40.0000   12.2222   10.0000
```

filter2(filter,image,'full')

returns a result larger than the original

Another approach to create filter.

Ex: use spatial function to create average filter

```
fspecial('average',[5,7])

fspecial('average',11)

fspecial('average` )
```

```
>> c=imread('cameraman.tif');
>> f1=fspecial('average');
>> cf1=filter2(f1,c);
```

To display the image we should follow steps:

1. **Convert image to the type uint8.**
2. **Divide all values on 255.**
3. **Apply (mat2gray) to scale the result for display:**

```
figure,imshow(c),figure,imshow(cf1/255)
```

Original image

Average filtering

Using a 9*9 filter

Using a25*25 filter

Separable filters

Some filters can be implemented by the successive application of two simpler filters.

Ex:

$$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} = \frac{1}{3}\begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \frac{1}{3}\begin{bmatrix} 1 & 1 & 1 \end{bmatrix}$$

The 3*3 averaging filter is thus separable into two smaller filters.

separately can result in great time savings.

Separable filters

for each pixel in the image.

n*n filter requires n^2 multiplications, and n^2-1 additions.

an n*1 filter only requires n multiplications and n-1 additions.

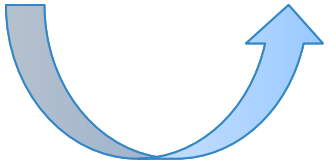
So, the total number of multiplications 2n and 2n-2 additions.

Separable filters-cont

- **Ex**

$$\begin{bmatrix} 1 & -2 & 1 \\ -2 & 4 & -2 \\ 1 & -2 & 1 \end{bmatrix} = \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & -2 & 1 \end{bmatrix}.$$

3*3 3*1 1*3



Frequencies low and high pass Filters.

There Is some Aspects Of An Image That aim Us To use appropriate Filter for a given image processing task.

- One important aspect of an image Is:: frequencies

- Frequency is measure Of Amount By Which Gray Value Change With Distance

1) High frequency components are characterized by large changes in grey values over small distances.

Example edges and noise.

2) Low frequency components are parts of the image characterized by little change in the grey values.

Example backgrounds, skin textures.

Frequencies low and high pass Filters.

- We then say that a Filter is ::
  1) High pass Filter if it "passes over "the high frequency components, and reduces or eliminates low frequency components.
     -- Example : laplacian Filter. Used For Edge Detection And Edge Enhancement.

  2) Low pass Filter if it "passes over" the low frequency components, and reduces or eliminates high frequency components.
     -- Example : Averaging Filter. Used To Blure Edges.

Frequencies low and high pass Filters.(Cont)

12/3/2025

Note

The sum of the coefficients (that is, the sum of all elements in the matrix), in the high pass Filter is

laplacian Filter

$$\begin{bmatrix} 1 & -2 & 1 \\ -2 & 4 & -2 \\ 1 & -2 & 1 \end{bmatrix}$$

- **Apply the above high pass Filter to:**

150	152	148	149
147	152	151	150
152	148	149	151
151	149	150	148

→

11	6
−13	−5

The resulting values are close to zero.

Using Camera Man Image Do This In Matlab g Camera Man Image Do This In Matlab

Laplacian Filter

```
>> f=fspecial('laplacian')

f =

    0.1667    0.6667    0.1667
    0.6667   -3.3333    0.6667
    0.1667    0.6667    0.1667

>> cf=filter2(f,c);
>> imshow(cf/100)
>> f1=fspecial('log')

f1 =

    0.0448    0.0468    0.0564    0.0468    0.0448
    0.0468    0.3167    0.7146    0.3167    0.0468
    0.0564    0.7146   -4.9048    0.7146    0.0564
    0.0468    0.3167    0.7146    0.3167    0.0468
    0.0448    0.0468    0.0564    0.0468    0.0448

>> cf1=filter2(f1,c);
>> figure,imshow(cf1/100)
```

High Pass Filter

Laplacian Filter

Laplacian of Gaussian (log) Filtering

Values outside the range 0-255

- **The result of applying a linear Filter may be values which lie outside The range 0-255 We consider dealing with This values.**
  - ✓ **Make negative values positive.**
  - ✓ **Clip values. We apply the following thresholding:**

$$y = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } 0 \leq x \leq 255 \\ 255 & \text{if } x > 255 \end{cases}$$

It is not suitable if there are many grey values outside the 0-255 rang

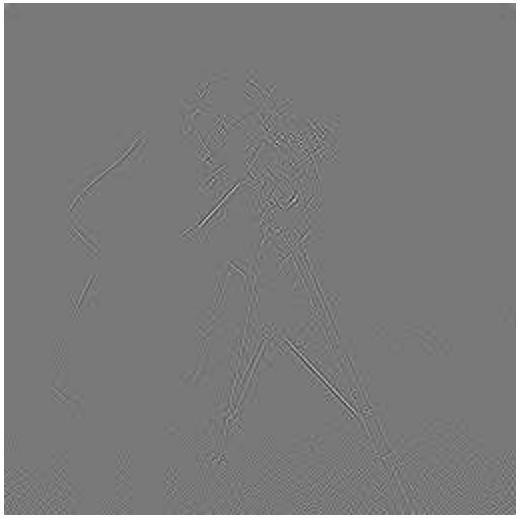
Values outside the range 0-255.(Cont)

- **Example: Using a high pass Filter and displaying the result**

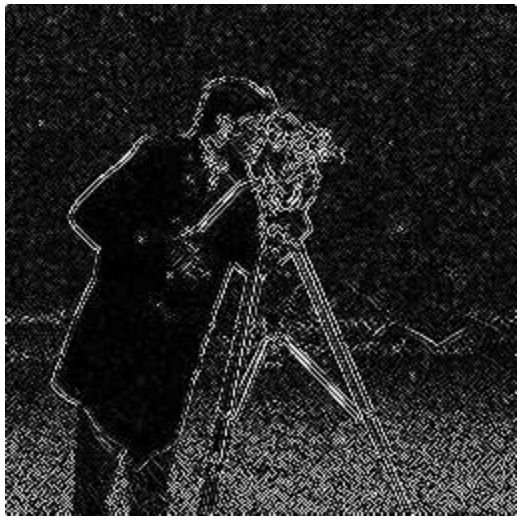
```
>> f2=[1 -2 1;-2 4 -2;1 -2 1];
>> cf2=filter2(f2,c);
```

```
>> figure,imshow(mat2gray(cf2));
```

```
>> figure,imshow(cf2/60)
```



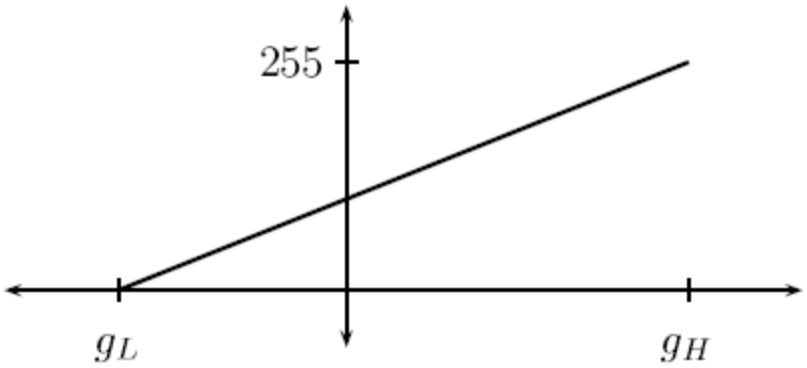
Using mat2gray



Dividing by a constant

Scaling transformation

- Lowest grey value produced by the Filter g_L.
- Highest value is g_H.
- We can transform all values in the range g_L-g_H by Linear Transformation.



$$y = 255\frac{x - g_L}{g_H - g_L}$$

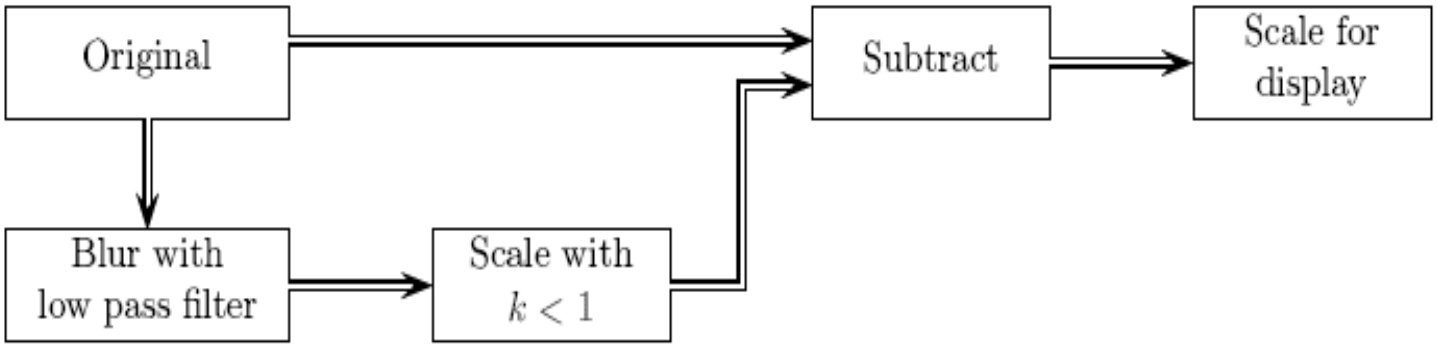
Edge sharpening

- can be used to make edges in an image slightly sharper and crisper, which generally results in an image more pleasing to the human eye.

This Operation Called :

- Edge enhancement = Edge crispening = unsharp masking.

Unsharp masking



Schema for unsharp masking

Edge sharpening.(Cont)

- **EX: Suppose an image is of type uint8. The unsharp masking can be applied by the following sequence of commands:**

```
>> f=fspecial('average');
>> xf=filter2(f,x);
>> xu=double(x)-xf/1.5
>> imshow(xu/70)
```

Edge sharpening.(Cont)

- **We can in fact perform the Filtering and subtracting operation in one command By :**

  using the linearity of the Filter, and that the 3*3 Filter.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

identity Filter

K: Constant To Provide Best Result

$$f = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} - \frac{1}{k}\begin{bmatrix} 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \end{bmatrix}$$

$$f = k\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} - \begin{bmatrix} 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \end{bmatrix}$$

unsharp masking can be implemented by a Filter of the form

Edge sharpening.(Cont)

- **The unsharp option of fspecial produces such Filters the Filter created has the form:**

$$\frac{1}{\alpha+1}\begin{bmatrix} -\alpha & \alpha-1 & -\alpha \\ \alpha-1 & \alpha+5 & \alpha-1 \\ -\alpha & \alpha-1 & -\alpha \end{bmatrix}$$

α: an optional parameter Its defaults = 0.2

If α =0.5

$$\frac{1}{3}\begin{bmatrix} -1 & -1 & -1 \\ -1 & 11 & -1 \\ -1 & -1 & -1 \end{bmatrix} = 4\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} - 3\begin{bmatrix} 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \end{bmatrix}$$

using the Matlab commands

```
>> p=imread('pelicans.tif');
>> u=fspecial('unsharp',0.5);
>> pu=filter2(u,p);
>> imshow(p),figure,imshow(pu/255)
```

Edge enhancement with unsharp masking

The original

After unsharp masking

High boost filtering

Allied to unsharp masking Filters are the high boost Filters, which are obtained by:

$$\text{high boost} = A(\text{original}) - (\text{low pass}).$$

Special Case: If A=1 the high boost Filter becomes an ordinary high pass Filter

If we take as the low pass Filter the 3*3 Averaging Filter

$$\frac{1}{9}\begin{bmatrix} -1 & -1 & -1 \\ -1 & z & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

If we put Z=11 we obtain a Filtering very similar to the unsharp Filter, except for a scaling factor

```
>> f=[-1 -1 -1;-1 11 -1;-1 -1 -1]/9;
>> xf=filter2(x,f);
>> imshow(xf/80)
```

High boost filtering (Cont)

Note

We can also write the high boost formula as:

$$\begin{aligned}\text{high boost} &= A(\text{original}) - (\text{low pass}) \\ &= A(\text{original}) - ((\text{original}) - (\text{high pass})) \\ &= (A-1)(\text{original}) + (\text{high pass}).\end{aligned}$$

❖**Best results for high boost Filtering are obtained if we multiply the equation by a factor (W)**

$$wA - w = 1$$

or

$$w = \frac{1}{A-1}.$$

So a general unsharp masking formula is

$$\frac{A}{A-1}(\text{original}) - \frac{1}{A-1}(\text{low pass}).$$

Another version of this formula is

$$\frac{A}{2A-1}(\text{original}) - \frac{1-A}{2A-1}(\text{low pass})$$

where for best results A is taken so that

$$\frac{3}{5} \leq A \leq \frac{5}{6}.$$

If we take $A = 3/5$, the formula becomes

$$\frac{3/5}{2(3/5)-1}(\text{original}) - \frac{1-(3/5)}{2(3/5)-1}(\text{low pass}) = 3(\text{original}) - 2(\text{low pass})$$

If we take $A = 5/6$ we obtain

$$\frac{5}{4}(\text{original}) - \frac{1}{4}(\text{low pass})$$

```
>>  id=[0  0  0;0  1  0;0  0  0];
>>  f=fspecial('average');
>>  hb1=3*id-2*f

hb1  =

    -0.2222    -0.2222    -0.2222
    -0.2222     2.7778    -0.2222
    -0.2222    -0.2222    -0.2222

>>  hb2=1.25*id-0.25*f

hb2  =

    -0.0278    -0.0278    -0.0278
    -0.0278     1.2222    -0.0278
    -0.0278    -0.0278    -0.0278
```

- **To show these results:**

x1=filter2(hb1,x); imshow(x1/255)

x2=filter2(hb2,x);imshow(x2/255)

High boost Filtering

High boost Filtering with hb1

High boost filtering with hb2

The Result Of Filter, hb1 appears to produce the best result Than hb2

Non-linear Filters

- **A non-linear Filter is obtained by a non-linear function of the grayscale values in the mask.**

Examples

1) ***Rank-order Filters:*** the elements under the mask are ordered.
   a) **maximum Filter:** its output the maximum value under the mask.
   b) **minimum Filter:** its output the minimum value under the mask.

If the values are given in increasing order, the minimum Filter is a rank-order Filter for which *the First element* is returned, and the maximum Filter is a rank-order Filter for which *the last element* is returned.

2) Geometric mean Filter.

Implementing *Rank-order Filters* in Mat lab

- **The function is (nlfilter)**

Ex: First to implement a maximum Filter over 3*3 neighborhood:

```
>> cmax=nlfilter(c,[3,3],'max(x(:))');
```

Using a maximum Filter

```
>> cmin=nlfilter(c,[3,3],'min(x(:))');
```

Using a minimum Filter

- **In each case the image has lost some sharpness, and has been brightened by the maximum Filter, Darkened by the minimum Filter.**

The nlfilter function is very slow.

A faster alternative is to use the **(colfilt)** function, which rearranges the image into columns First.

Ex:

```
>> cmax=colfilt(c,[3,3],'sliding',@max);
```

The parameter sliding indicates that overlapping neighborhoods are being used.

Implement the max and min Filters as rank-order Filters

- The function Is (ordfilt2) .
- This requires three inputs Parameters:

1) The image,
2) The index value of the ordered results to choose as output,
3) The definition of the mask.

Ex:

```
>> cmax=ordfilt2(c,9,ones(3,3));
```

The Max Filter

```
>> cmin=ordfilt2(c,1,ones(3,3));
```

The Min Filter

```
>> cmed=ordfilt2(c,5,ones(3,3));
```

The Median Filter

Geometric mean Filter

- **It's another non-linear Filters which is defined as:**

$$\left(\prod_{(i,j)\in M} x(i,j) \right)^{(1/|M|)}$$

M:Filter Mask
|M|:Its Size

Alpha-Trimmed Mean Filter Steps:

1) Orders the values under the mask.
2) Trims Off elements at either end of the ordered list.
3) Takes the mean of the remainder.

Ex:

$$x_1 \leq x_2 \leq x_3 \leq \cdots \leq x_9$$

and trim off two elements at either end, the result of the filter will be

$$(x_3 + x_4 + x_5 + x_6 + x_7)/5.$$

Thank You